

Intelligenza Artificiale

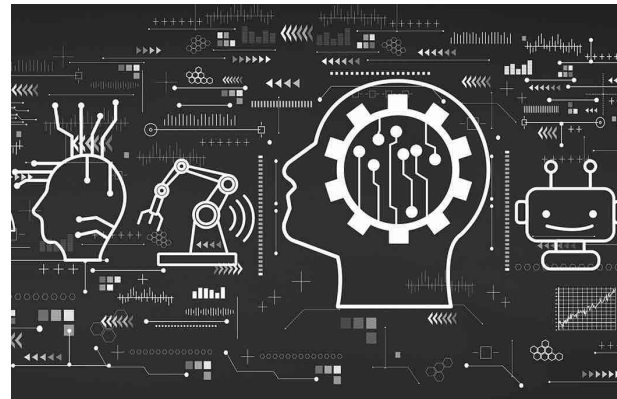
“Machine learning (part 2)”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

- Loss function
- Linear regression
- Gradient descent
- Classification
- Support Vector Machines



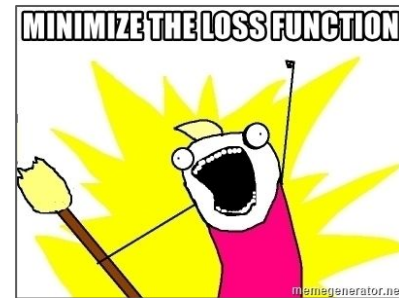
Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

From error rates to loss

- In ML we often minimize a **loss function** rather than maximize a utility
- The loss function $L(x, y, \hat{y}) \sim L(y, \hat{y})$ is defined as the amount of utility lost by predicting $h(x) = \hat{y}$ when the correct answer is $f(x) = y$

$$L(x, y, \hat{y}) = \text{Utility}(\text{result of using } y \text{ given an input } x) \\ - \text{Utility}(\text{result of using } \hat{y} \text{ given an input } x)$$

- The learning agent maximizes its expected utility by choosing the hypothesis that minimizes expected loss over all input-output pairs it will see
- Two common types of loss
 - **Absolute-value loss** (L_1 loss) $L_1(y, \hat{y}) = |y - \hat{y}|$
 - **Squared-error loss** (L_2 loss) $L_2(y, \hat{y}) = (y - \hat{y})^2$



Loss in Linear Regression

- E.g. **univariate linear regression**

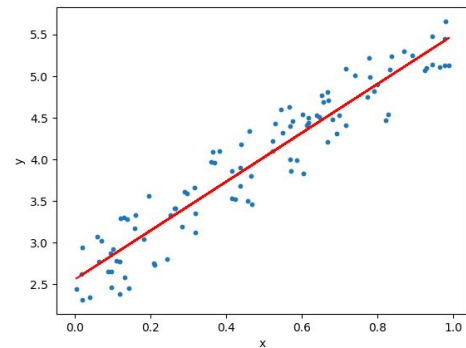
- Input x and output y
- Hypothesis: **linear function**

$$h_{\mathbf{w}}(x) = w_1 x + w_0$$

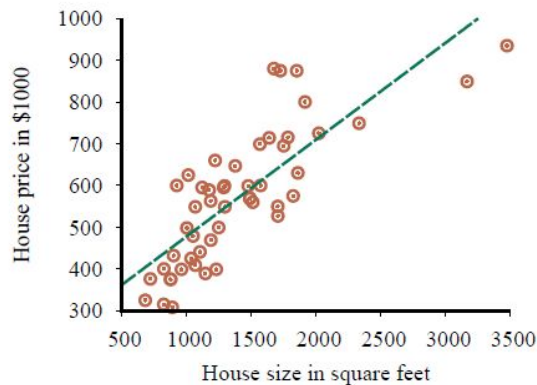
- **TASK:** finding $h_{\mathbf{w}}$ that best fits the data

⇒ find the values of the weights (w_0, w_1) that minimize squared-error loss function L_2 , summed over all the N training examples:

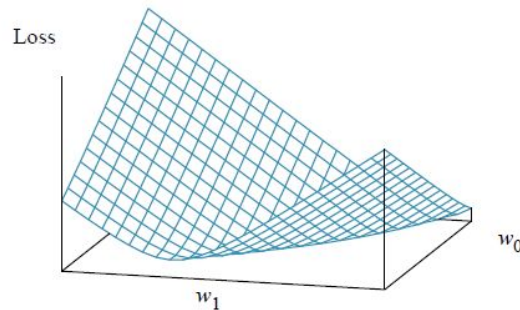
$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^N L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^N (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2$$



Example of univariate linear regression



(a)



(b)

- (a) E.g. data points of price vs floor space of houses for sale in Berkeley + linear function hypothesis that minimizes squared-error loss L_2
- (b) Plot of loss function $\sum_j (y_j - (w_1 x_j + w_0))^2$ for various values of w_0 , w_1 , i.e. a **weight space**

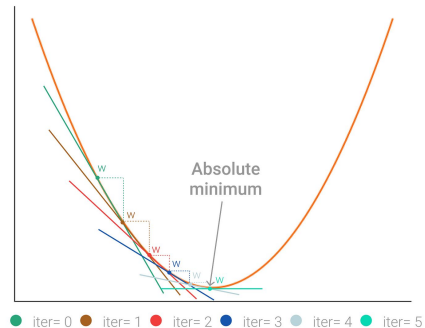
Note that the loss function is convex with a global minimum in the parameters of the regression line

Gradient descent

- Finding the minimum loss for univariate linear case is easy
 - There is an analytical solution for when the partial derivatives in w_0 and w_1 are $= 0$
- For more complex cases, an approximate solutions can be obtained with a method similar to **hill climbing**, but instead of maximizing a gain, we minimize a loss – process called **gradient descent**
- That is, search through a continuous weight space by incrementally modifying the parameters:

$\mathbf{w} \leftarrow$ any point in the parameter space
while not converged do
 for each w_i in $\mathbf{w}$ do
 $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} Loss(\mathbf{w})$

- α is called **step size** or **learning rate** (fixed or decay over time)



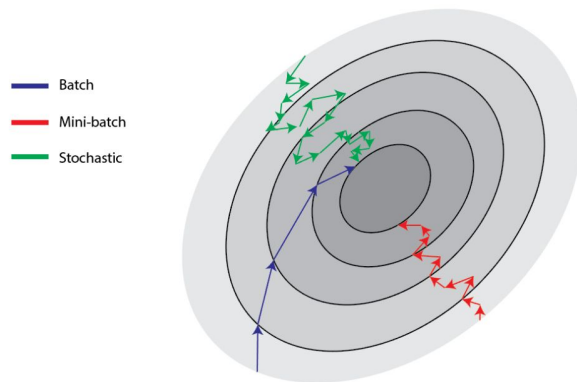
Batch gradient descent

- The previous update was for a single sample, but for N samples we need to minimize the sum of all the N individual losses

$$w_i \leftarrow w_i - \alpha \sum_j \partial \text{Loss}_j / \partial w_i$$

where $\text{Loss}_j = L_2(y_j, h_w(x_j))$, for $j = 1 \dots N$

- This is the **batch gradient descent**
- Since loss surface is convex, convergence to global minimum is guaranteed
 - as long α is not too large, otherwise the update overshoots
- A step that covers all the training examples is called an **epoch**



- **Stochastic gradient descent (SGD)**
 - randomly selects only one training example j at each step (faster)
- Other variant is **minibatch SGD** – randomly select a minibatch of m out of the N examples
 - We could choose m to take advantage of hardware parallel vector operations

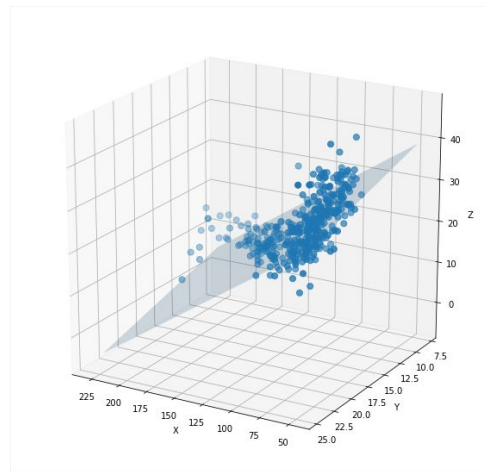
Multivariable linear regression

- $\mathbf{x}_j$ is an input **n -element vector** and the hypothesis is the dot product of the weights $\mathbf{w}$ and $\mathbf{x}_j$

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i}$$

- $\mathbf{w}^*$ is the best vector of weights that minimizes the squared-error loss

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j)$$



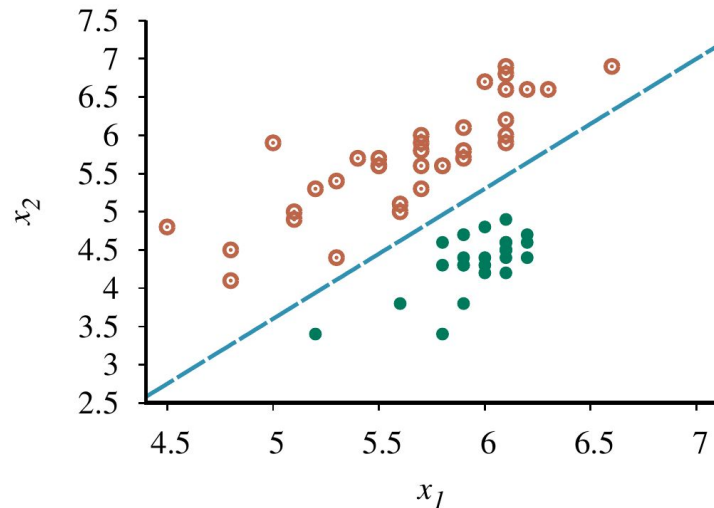
- Gradient descent will reach the (unique) minimum of the loss function

Linear regression and classification

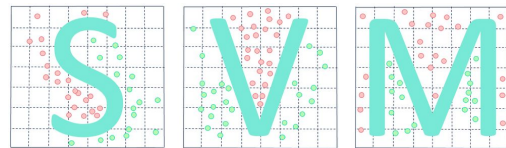
- Linear functions can be used to do classification as well as regression
- **Decision boundary**: a line (or a surface, in higher dimensions) that separates the two classes
 - **Linear separator**: linear decision boundary for linearly separable data
- h : result of passing the linear function $\mathbf{w} \cdot \mathbf{x}$ through a **threshold function**:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x})$$

where $\text{Threshold}(z) = 1$ if $z \geq 0$ and 0 otherwise



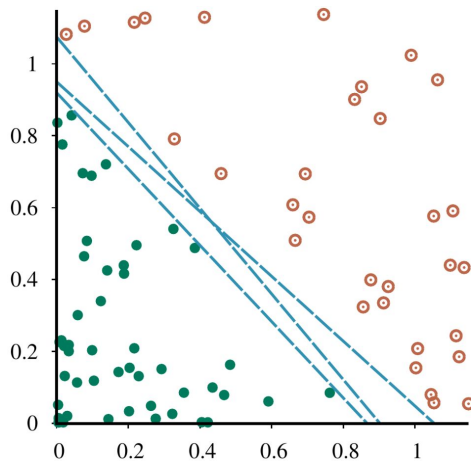
Support vector machines



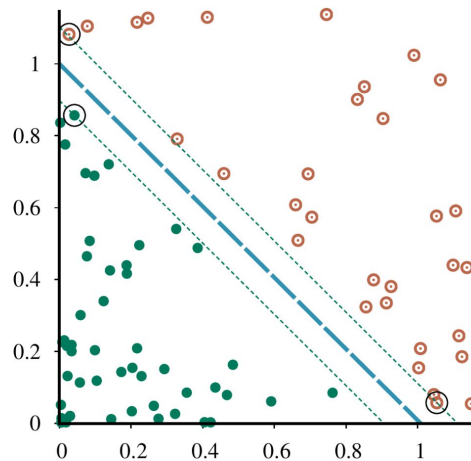
- SVMs have three attractive properties:
 - SVMs construct a **maximum margin separator**
 - i.e. a decision boundary with the largest possible distance to example points
 - SVMs create a **linear separating hyperplane**
 - SVMs are **nonparametric**
 - the separating hyperplane is defined by a set of examples

Support vectors

- (a) Two classes of points and three candidate linear separators: which one should we choose?
- (b) The **maximum margin separator** (heavy line) is at the midpoint of the margin (area between dashed lines)
 - The **support vectors** (large circles) are the examples closest to the separator, 3 in this case



(a)

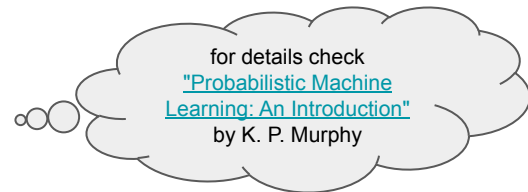


(b)

SVM classification

- The separator between points is defined as the set of points
- Optimal solution is found by solving the **dual problem**

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$

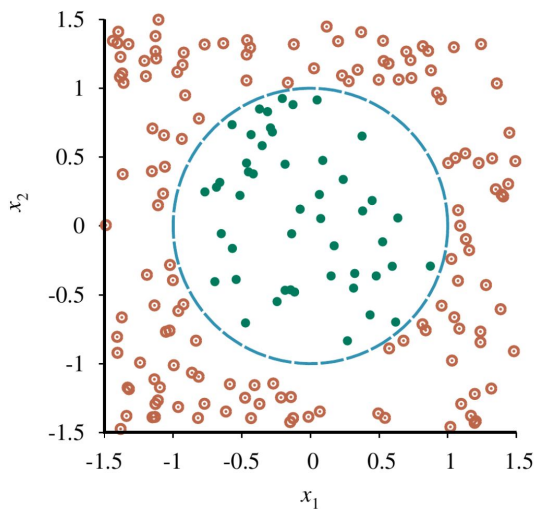


- The weights α_j associated with each data point are zero except for the **support vectors** – the points closest to the separator
- Expression is convex – it has a single global maximum that can be found efficiently
- Once the optimal weights α_j have been found, the classification is done on a **0** threshold

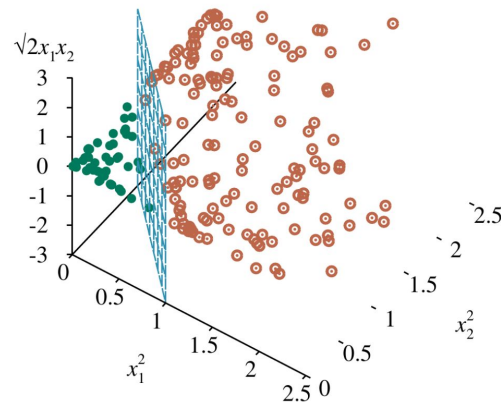
$$h(\mathbf{x}) = \operatorname{sign} \left(\sum_j \alpha_j y_j (\mathbf{x} \cdot \mathbf{x}_j) - b \right)$$

Example: SVM classification in nonlinear case

- (a) A two-dimensional training set with positive and negative examples
- (b) The same data after mapping into a three-dimensional input space – the circular decision boundary becomes a linear decision boundary in three dimensions



(a)



(b)

SVM kernel trick



- No need to convert inputs to higher dimensional spaces
- We can just replace $\mathbf{x}_j \cdot \mathbf{x}_k$ with a **kernel function** $K(\mathbf{x}_j, \mathbf{x}_k)$

$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$

where $K(\mathbf{x}_j, \mathbf{x}_k) = F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)$ for some high-dimensional transformation F

- We say that the algorithm has been “**kernelized**”
- Plugging kernel into equation, optimal linear separators (in high-dimensional spaces) can be found
 - The resulting linear separators, when mapped back to the original input space, could correspond to arbitrarily wiggly, nonlinear decision boundaries
- Common kernels include polynomial $(c + \mathbf{x}_j \cdot \mathbf{x}_k)^d$ and Gaussian $\exp(-\gamma \|\mathbf{x}_j - \mathbf{x}_k\|^2)$

Conclusions

- Suggested reading
 - Chapter 19, sec. 19.4, 19.6, 19.7.5 of Russell & Norvig
- Next lecture
Deep learning (part 1)
- Questions?

